كلية العلوم التطبيقية - أيت ملول
FACULTE DES SCIENCES APPLIQUEES – AIT MELLOUL

Programmation Python
Filières AD & SIE
S5
2025/2026

TP2

Gestion d'un Système de Fichiers d'une Petite Entreprise

➢ Objectifs pédagogiques

- Manipuler des **fichiers textes** et **fichiers structurés (CSV simple, JSON simple)** sans bibliothèques externes.
- Gérer des **exceptions** dans un programme Python complexe.
- Organiser un programme en **fonctions** et en **classes** (OOP).
- Manipuler le **système de fichiers** : création, suppression, déplacement, lecture, écriture.
- Structurer et valider des données provenant de fichiers.

➢ Contexte du TP

Vous devez développer un **mini-système de gestion des employés et des départements** pour une petite entreprise.

L'entreprise stocke ses données sous forme de **fichiers texte**, et souhaite automatiser plusieurs opérations :

- Création et lecture de fichiers employés
- Mise à jour des informations
- Recherche d'employés
- Gestion des erreurs liées au système de fichiers
- Manipulation de fichiers structurés
- Exportation au format JSON (sans librairies externes)

Votre rôle est d'écrire un programme Python robuste, organisé et sécurisé avec des exceptions personnalisées.

➢ Fichiers fournis (à générer avant de commencer le TP)

1. employees.txt

```
1001;Ahmed Soulaimane;Informatique;4500
1002;Othmane El Idrissi;Comptabilite;3800
1003;Sara Ben Ali;Marketing;4200
1004;Kamal Jaziri;Informatique;5000
```

Format :

ID ; Nom complet ; Département ; Salaire

2. departments.txt

```
Informatique
Marketing
Comptabilite
RessourcesHumaines
```

➢ Travail demandé

PARTIE 1 — Manipulation des fichiers texte

Exercice 1 : Lecture robuste

Écrire une fonction :

```
def load_employees(file_path):
    ...
```

qui :

- Lit le fichier line par line,
- Vérifie la validité du format (4 champs séparés par ;),
- Ignore les lignes incorrectes,
- Utilise au moins **3 types** différents d'exceptions (**FileNotFoundError**, **ValueError**, **IndexError**, etc.),
- Retourne une liste d'employés sous forme de dictionnaires :

```
{
    "id": int,
    "name": str,
    "department": str,
    "salary": float
```

```
}
```

Exercice 2 : Validation des départements

Écrire une fonction :

```
def load_departments(file_path):
  ...
```

qui lit **departments.txt** et retourne une **liste**.

Gérer les exceptions suivantes :

- Fichier manquant,
- Fichier vide,
- Lignes contenant des caractères invalides.

Exercice 3 : Filtrage et recherche

Écrire les fonctions :

```
def find_employee_by_id(employees, emp_id)
def filter_by_department(employees, department)
```

Gérer les cas d'erreurs (ID non entier, département inexistant…).

PARTIE 2 — POO et gestion du système de fichiers

Exercice 4 : Création d'une classe EmployeeManager

Créer une classe :

```
class EmployeeManager:
  def __init__(self, employees, departments): ...
```

Méthodes attendues :

- ***add_employee(emp_id, name, department, salary)***,
- ***remove_employee(emp_id)***,
- ***update_salary(emp_id, new_salary)***,
- ***save_to_file(path)*** → réécrit employees.txt.

Chaque méthode doit :

- Lever des ***exceptions personnalisées*** (ex. **InvalidDepartmentError**, **EmployeeNotFoundError**),
- Capturer toutes les autres exceptions possibles,
- Garantir la cohérence des données.

Exercice 5 : Manipulation du système de fichiers

Écrire les fonctions suivantes :

```
def backup_file(src_path, backup_folder):
    ...
```

Fonctionnalités :

- Créer le dossier de backup s'il n'existe pas,
- Copier le fichier employees.txt sous un nom du type : **employees_backup_YYYYMMDD_HHMM.txt**
- Gérer les exceptions :
  - Permissions,
  - Fichier introuvable,
  - Impossibilité de créer le dossier.

PARTIE 3 — Fichiers structurés (CSV simple et JSON)

Exercice 6 : Export au format CSV simple

Créer la fonction :

```
def export_csv(employees, path):
    ...
```

Format CSV (séparateur virgule) :

```
id,name,department,salary
1001,Ahmed Soulaimane,Informatique,4500
...
```

Gérer les erreurs :

- Permission refusée,
- Données manquantes,
- Valeurs non convertibles en texte.

Exercice 7 : Export au format JSON (sans bibliothèques externes)

Créer la fonction :

```
def export_json(employees, path):
    ...
```

Contraintes :

- Ne pas utiliser la bibliothèque json
- Ecrire manuellement un fichier JSON valide :

```
{
  "employees": [
```

```
        {
          "id": 1001,
          "name": "Ahmed Soulaimane",
          "department": "Informatique",
          "salary": 4500
        },
        ...
      ]
    }
```

Cas à gérer :

- Caractères qui nécessitent un échappement,
- Erreurs d'écriture,
- Liste vide → créer un JSON correct.

PARTIE 4 — Exceptions personnalisées

Créer un fichier exceptions.py contenant :

- class **EmployeeError**(Exception)
- class **EmployeeNotFoundError**(EmployeeError)
- class **InvalidDepartmentError**(EmployeeError)
- class **InvalidSalaryError**(EmployeeError)
- class **FileBackupError**(EmployeeError)

Chaque exception doit contenir :

- Message personnalisé,
- Représentation sous forme de chaîne (__str__).

➢ Travail à rendre

Un dossier projet contenant :

- main.py,
- exceptions.py,
- employees.txt et departments.txt,
- Tous les fichiers générés (CSV, JSON, backup).